Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Algorithmic Maze Simulator:

Team Name: *Team #*	*HUSTLERS* *DSCPP-III-2026-2338*
Team member 1 (Team Lead)	 **Sharma Jaydeep** Student ID: 2027835 2510010920@geu ac in
Team member 2	 **Singh Jaskirat** Student ID: 2027834 2510010581@geu ac in

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

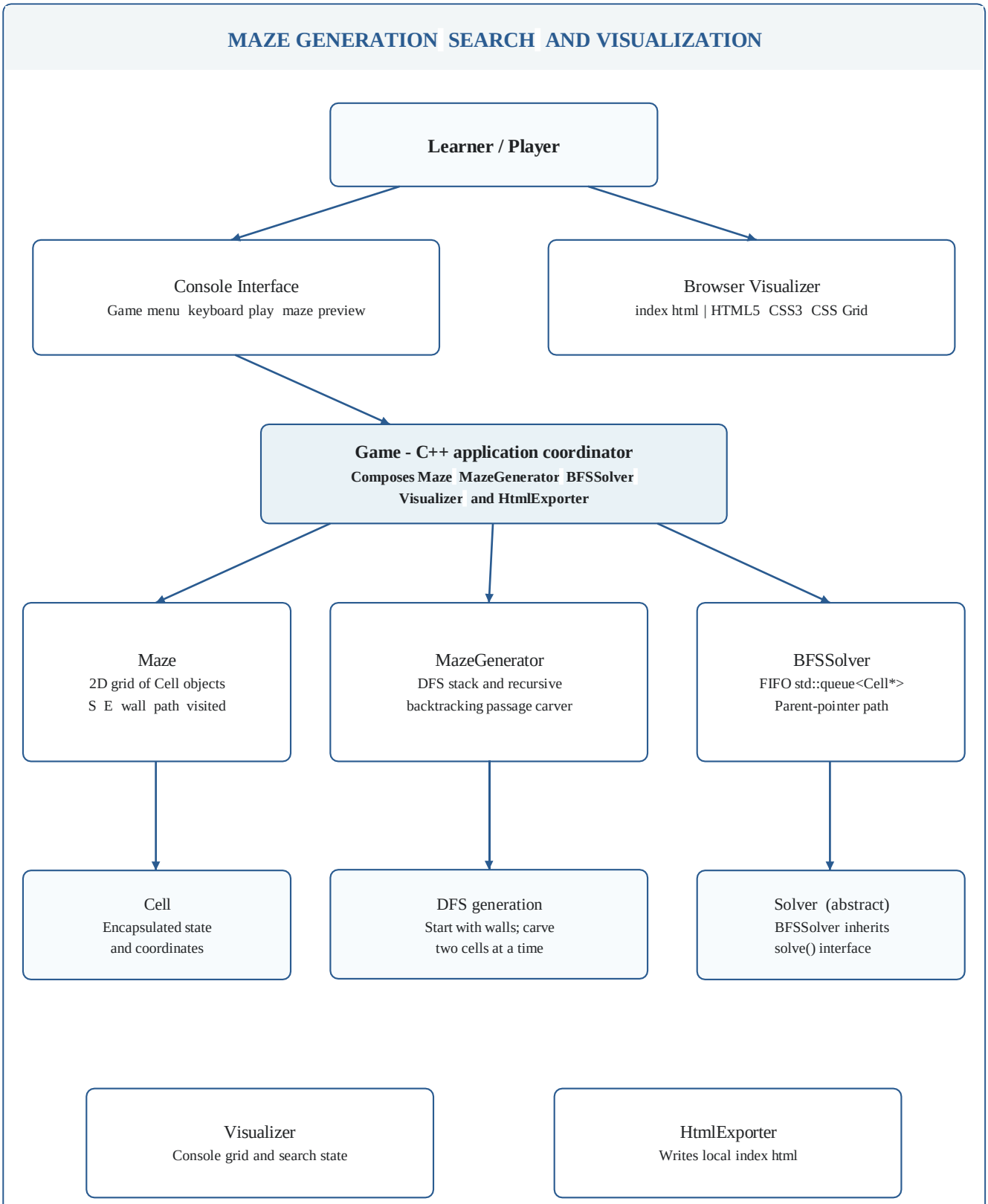
Algorithmic Maze Simulator is a third-semester DSA and OOP project that generates solvable mazes and demonstrates shortest-path search A C++ Standard Library application uses recursive depth-first search (DFS) backtracking to carve passages and breadth-first search (BFS) with a FIFO queue and parent pointers to recover a shortest route from S to E It provides console play three preset levels regeneration BFS solving maze preview and export to an HTML/CSS visualizer The technical description specifies a CSS Grid interface with state tabs and no JavaScript or external server Supplied screenshots show a newer guided interface with custom setup and five solver choices; those extra modes are distinguished from the code features confirmed in the written description

Updated Project Approach and Architecture

The C++ application centers on **Game** which composes **Maze MazeGenerator BFSSolver** and **Visualizer Maze** owns a 2D grid of **Cell** objects; cells encapsulate states such as WALL PATH START END VISITED and SOLUTION The generator begins with walls and carves passages two cells at a time using DFS and recursive backtracking **BFSSolver** derives from abstract **Solver** explores cells level by level with std::queue<Cell*> and follows parent pointers from E back to S to reconstruct a shortest route BFS runs in O(V+E) or O(R x C) on this grid with O(V) space

The console menu supports manual WASD/arrow-key play with wall collision difficulty selection regeneration BFS solving HTML export grid preview and exit Windows builds use MinGW/g++ via **build bat** or **make** and produce **bin/maze_game exe** The local visualizer uses HTML5 Vanilla CSS3 CSS Grid and CSS state tabs; its controls switch the three levels and toggle the BFS path No external web server is required The screenshots additionally display custom-size mode benchmark and leaderboard controls; their integration status is not verified by the technical description

System Architecture (High-Level Diagram)



Generation: the grid begins as walls; DFS carves passages two cells at a time to preserve partitions and guarantees a route from S to E BFS explores distance layers in order and reconstructs the shortest path by following parents backward from E then reversing the route **Complexity:** O(V+E) equivalent to O(R x C) for this grid; O(V) queue and visited space **OOP:** Maze contains Cells Game composes its modules and BFSSolver inherits the abstract Solver

Interface Screenshots and Solver Modes

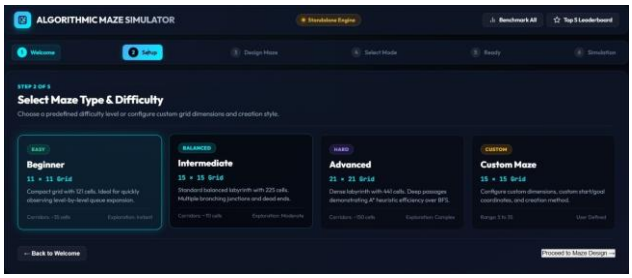
Cropped from the screenshots supplied with the project

Welcome and Feature Summary



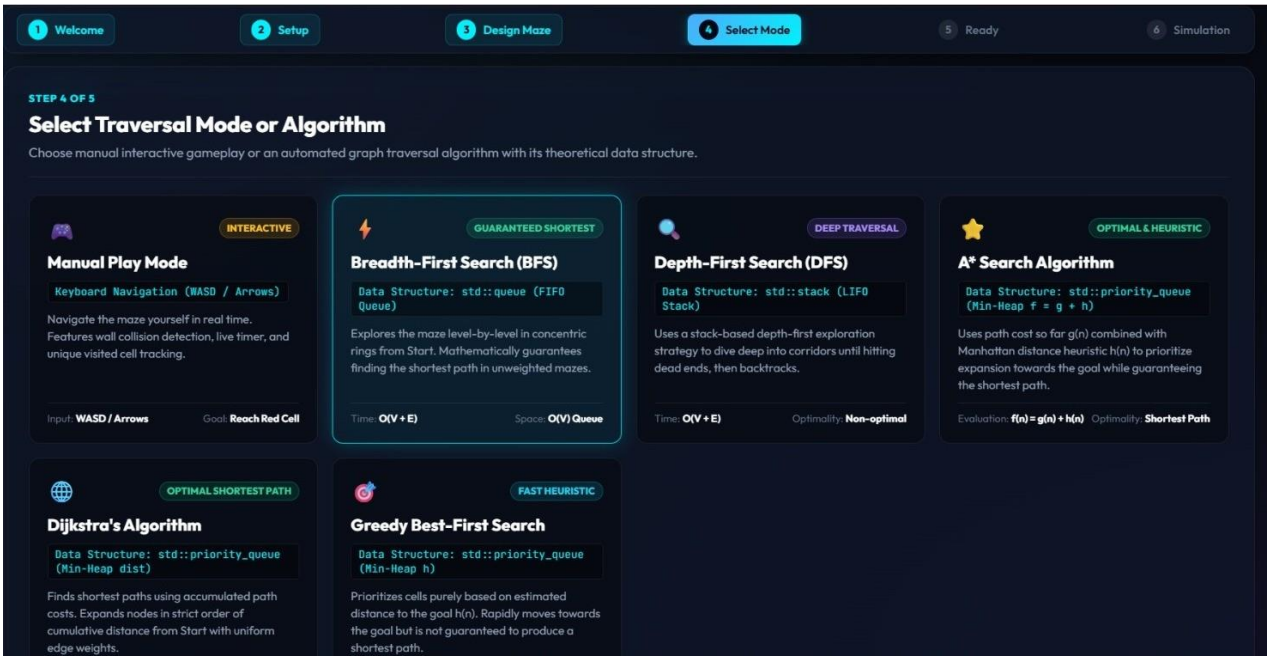
1 Welcome screen: feature cards Benchmark All Top 5 Leaderboard

Setup and Difficulty Selection



2 Setup: Easy 11x11 Medium 15x15 Hard 21x21 Custom (5-35)

Mode Selection and Algorithm Details



The visible selector offers manual play plus BFS DFS A* Dijkstra and Greedy Best-First

The setup image also shows a custom maze card with adjustable dimensions start/goal coordinates and a creation method The welcome screen shows five pathfinding solvers a custom maze studio manual WASD gameplay benchmarking and a persistent top-five ranking

Mode	Data structure / rule	Behavior described on screen
Manual play	WASD / arrow keys	Real-time movement wall collision timer visited-cell tracking
BFS	FIFO queue	Layer-by-layer search; shortest path in an unweighted grid
DFS	LIFO stack	Deep exploration with backtracking; not always shortest
A*	Min-heap; f = g + h	Manhattan heuristic; shortest-path claim shown in the UI
Dijkstra	Min-heap; accumulated distance	Expands in increasing path cost; optimal for uniform weights
Greedy Best-First	Min-heap; h	Moves toward the goal heuristically; no shortest-path guarantee

Tasks Completed

Task Completed	Team Member
Created the C++ class structure for Game Maze Cell MazeGenerator Solver/BFSSolver and Visualizer	Jaydeep Sharma
Built randomized solvable mazes with DFS recursive backtracking and two-cell passage carving	Jaydeep Sharma
Implemented BFS with a FIFO queue and parent-pointer reconstruction of the shortest path	Jaydeep Sharma
Added console menu functions for manual WASD/arrow play difficulty selection regeneration BFS solving preview and HTML export	Jaydeep Sharma; Jaskirat Singh
Built a CSS-only browser visualizer with CSS Grid three levels CSS state tabs and a BFS path toggle	Jaskirat Singh

Challenges/Roadblocks

Maze generation must carve connected corridors while preserving the wall partitions; the two-cell DFS carve is designed to keep a route from S to E BFS must mark cells assign parent pointers and reconstruct the route in reverse order Console movement must block walls and recognize arrival at the goal; the earlier progress report notes that goal-detection errors were still being fixed The CSS-only visualizer must keep level and solution-path controls predictable without JavaScript The supplied materials also describe different scopes: the detailed project brief confirms BFS and three preset levels while an earlier draft mentions additional levels and solvers and the screenshots show more modes custom setup benchmarking and a leaderboard The team will reconcile the current build with these descriptions and test each displayed feature before marking it complete

Tasks Pending

Task Pending	Team Member (to complete the task)
Reconcile the five-mode selector with the executable; verify DFS and implement/test A* Dijkstra and Greedy Best-First if they are not integrated	Jaydeep Sharma
Verify custom dimensions start/goal placement and a reachability check for the maze editor	Jaydeep Sharma
Regression-test wall collision goal detection level changes regeneration and the BFS path toggle	Jaydeep Sharma; Jaskirat Singh
Verify Benchmark All and Top 5 Leaderboard behavior including saved scores and repeatable metrics	Jaskirat Singh
Test all three preset levels and record path length explored cells and run time; document the result	Jaydeep Sharma
Prepare the final demo and update the report after integration and testing	All listed team members

Project Outcome/Deliverables

The project demonstrates how DSA and OOP can solve a path-finding problem Its core deliverables are a C++ maze generator using DFS backtracking a BFS solver that returns a shortest route in an unweighted grid a keyboard-playable console mode and a local HTML/CSS visualizer The project includes three difficulty levels (11x11 15x15 and 21x21) a maze preview regeneration and an exportable BFS solution view The supplied interface also shows a guided setup custom maze controls five solver choices benchmarking and a top-five leaderboard The simulator supports learning about stacks queues grids recursion backtracking shortest paths and class design

Progress Overview

The earlier Phase-II draft estimated about 70% completion and recorded a successful build maze generation for 11x11/15x15/21x21 a Level 1 BFS solve in 25 steps and manual controls with goal-detection fixes still needed The detailed project brief now defines three preset levels and BFS as the verified solver It also specifies a CSS-only visualizer The supplied screenshots show a fuller wizard custom setup and five solver cards plus Benchmark All and Top 5 Leaderboard controls An earlier draft mentions DFS testing and 31x31/41x41 sizes; these claims do not match the detailed brief and are treated as unverified until checked against the current build The next stage is integration repeatable tests for each visible option goal-detection fixes and a documented final demonstration The custom editor was previously reported behind schedule; no dated schedule was supplied

Codebase Information

Repository: https://github com/sen339853-rgb/ALGORITHMIC-MAZE-SIMULATOR- git
Branch: CS
Important commits recorded: Initial Maze Cell and Solver structure; DFS maze generation; BFS/DFS solver work manual play and live statistics
Build and run: Windows with MinGW/GCC g++; run \build bat or make then run bin\maze_game exe The detailed brief specifies this output path; the earlier draft names maze_game exe without a folder
Web: local index html HTML5 Vanilla CSS3 CSS Grid and CSS state tabs; no JavaScript or external server per the detailed brief

Testing and Validation Status

Test Type	Status	Notes
Compilation	Pass (reported)	Earlier report records a successful build; current brief names bin/maze_game exe
Maze generation	Pass (reported)	11x11 15x15 and 21x21 mazes are recorded as generated
BFS shortest path	Pass (reported)	Level 1 solved in 25 steps and shown on the grid; BFS is shortest for unweighted grids
DFS solver	Confirm	Earlier draft reports a valid route not always shortest; detailed brief documents BFS only
Manual play	Partial	Movement controls respond; goal-detection errors were still being fixed
Browser / mode controls	Not tested	Screenshots show the UI; no functional test record was supplied for CSS toggles custom mode other solvers benchmark or leaderboard

Deliverables Progress

Completed in the written specification: DFS maze generation; BFS queue solver and shortest-path reconstruction; three preset levels; console play; local HTML/CSS visualizer and BFS path toggle

Shown in supplied interface: six-step flow custom maze controls manual play five solver choices Benchmark All and Top 5 Leaderboard Their code-level status is unverified

In progress / pending: resolve documentation differences test goal detection and every visible mode verify custom mazes and ranking persistence record benchmark results then complete the final demo